

FIAP GRADUAÇÃO

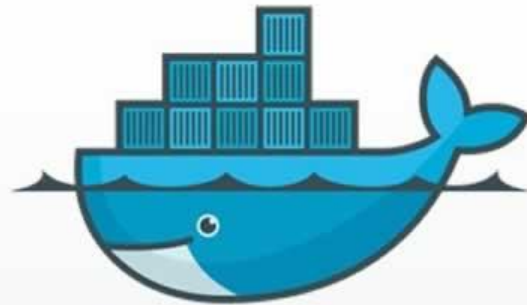
TECNOLOGIA EM DESENVOLVIMENTO DE SISTEMAS

DevOps Tools & Cloud Computing

Dockerfile

PROF. João Menk

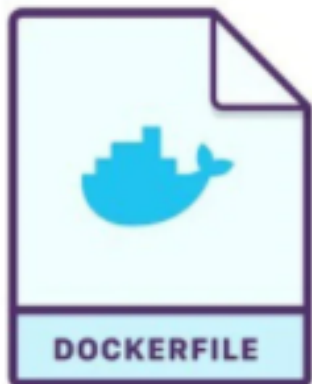
profjoao.menk@fiap.com.br



docker

DOCKERFILE

- A interface de linha de comando (CLI) é uma forma manual de realizar a administração. Visto em nossos exemplos não é complexo realizar comandos como: pull, run, ps, stop etc.
- Porém podemos automatizar o processo utilizando Dockerfiles. Esses arquivos nada mais são do que listas de instruções utilizados para automatizar a criação e configuração de Containers
- Em outras palavras, ele serve como **uma receita para construir uma Imagem**, permitindo definir um ambiente personalizado



No exemplo abaixo temos o fonte de um Dockerfile simples, que realiza alguns dos passos que já executamos em nossos exemplos

```
FROM ubuntu
```

Obter uma imagem do Ubuntu

```
RUN apt-get -y update
```

Atualizar os pacotes do Sistema Operacional da imagem

```
RUN apt-get -y install python
```

Instalar o Python

Instrução FROM

É obrigatória e define qual será o ponto de partida da Imagem que criaremos com o nosso Dockerfile

Instrução RUN

Pode ser executada uma ou mais vezes e, com ela, podemos definir quais serão os comandos executados na **etapa de criação de camadas da Imagem**

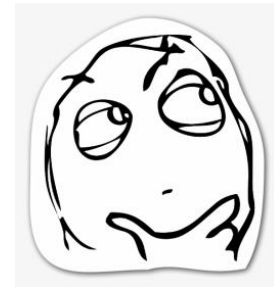
Crie um arquivo com o fonte do exemplo anterior em sua Home, no Sistema Operacional, e salve como **Dockerfile** (sem extensão) 

```
 Dockerfile X  
Users > Menk >  Dockerfile > ...  
1 FROM ubuntu  
2  
3 RUN apt-get -y update  
4 RUN apt-get -y install python3
```

Agora que escrevemos um Dockerfile, **iremos construir uma imagem** a partir desse arquivo, executando o comando **docker build**, e, por fim, criar e rodar um Container com o comando **docker container run**

“O Container é o fim enquanto a Imagem é o meio”

*#fica
a
dica*



Caso queira criar uma imagem do zero, sem a preocupação de utilizar imagem alguma, utilize a imagem **scratch**

FROM scratch

Para criar uma Imagem a partir desse arquivo usamos o comando **docker build**. Por padrão esse comando procura um arquivo com o nome **Dockerfile**

Em nosso exemplo o comando ficará assim:

docker build -t ubuntu_python .  **Diretório corrente (não esqueça do espaço antes do ponto)**

```
iMac$ docker build .
[+] Building 45.1s (7/7) FINISHED
=> [internal] load build definition from Dockerfile                                0.1s
=> => transferring dockerfile: 108B                                              0.0s
=> [internal] load .dockerignore                                                  0.1s
=> => transferring context: 2B                                                  0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest                 1.6s
=> [1/3] FROM docker.io/library/ubuntu@sha256:4b1d0c4a2d2aaf63b37111f34eb9      8.0s
=> => resolve docker.io/library/ubuntu@sha256:4b1d0c4a2d2aaf63b37111f34eb9    0.0s
=> => sha256:4b1d0c4a2d2aaf63b37111f34eb9fa89fa1bf53dd6e4c 1.42kB / 1.42kB    0.0s
=> => sha256:817cfe4672284dcbee885b1a66094fd907630d610cab3291 529B / 529B   0.0s
=> => sha256:a8780b506fa4eeb1d0779a3c92c8d5d3e6a656c758135 1.46kB / 1.46kB  0.0s
=> => sha256:e96e057aae67380a4ddb16c337c5c3669d97fdff69e 30.43MB / 30.43MB 5.7s
=> => extracting sha256:e96e057aae67380a4ddb16c337c5c3669d97fdff69ec537f02 1.5s
=> [2/3] RUN apt-get -y update                                                  22.3s
=> [3/3] RUN apt-get -y install python3                                         12.6s
=> exporting to image                                                            0.4s
=> => exporting layers                                                            0.4s
=> => writing image sha256:967ab74e1a522c005f30a5aea254dff474038e494ee7d6c 0.0s

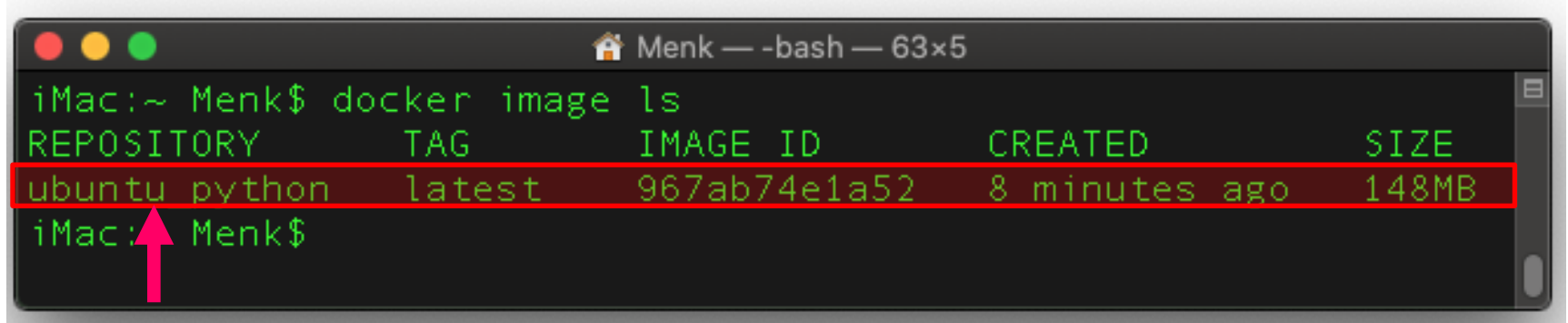
Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
iMac$
```

Caso necessite, especifique o nome do seu Dockerfile com a opção **-f**

Exemplo: **docker build -t ubuntu_python -f dockerfile-ubuntu-python .**

Verifique o resultado

docker image ls



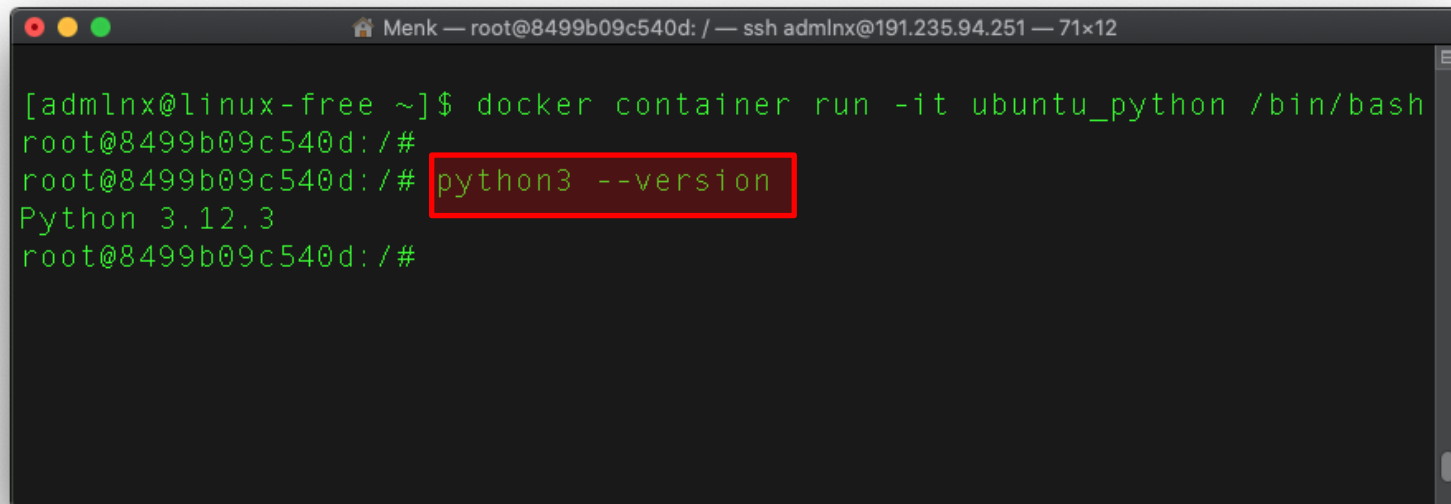
```
iMac:~ Menk$ docker image ls
REPOSITORY      TAG          IMAGE ID       CREATED        SIZE
ubuntu python   latest       967ab74e1a52   8 minutes ago 148MB
iMac:~ Menk$
```

Como último passo vamos rodar um Container criado por meio de um Dockerfile

Vamos rodar esse Container em modo Interativo e acessar o terminal para verificar se as tarefas foram concluídas

docker container run --rm -it ubuntu_python /bin/bash

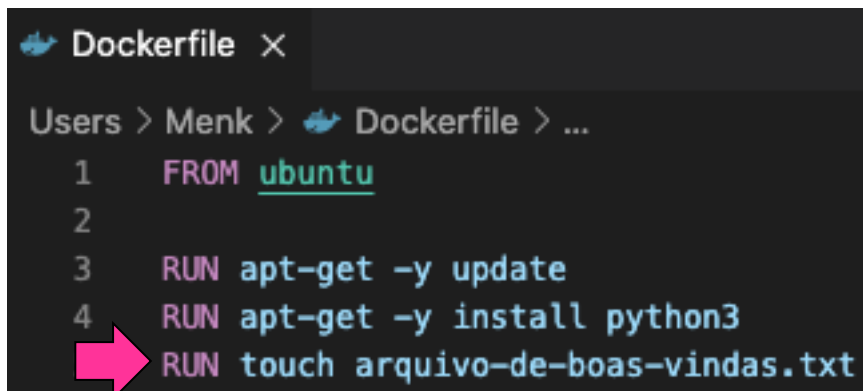
O Docker remove automaticamente o Container quando for encerrado



```
Menk — root@8499b09c540d: / — ssh admlnx@191.235.94.251 — 71x12
[admlnx@linux-free ~]$ docker container run -it ubuntu_python /bin/bash
root@8499b09c540d:/#
root@8499b09c540d:/# python3 --version
Python 3.12.3
root@8499b09c540d:/#
```

DOCKERFILE

- Agora vamos **recrir** a imagem a partir do Dockerfile alterando com novas instruções
- Cada RUN criará uma etapa na criação da Imagem
- Cada camada gerada por ele poderá ser reutilizada na criação de outras Imagens
- Altere seu Dockerfile conforme abaixo adicionando mais uma tarefa, salve e execute novamente o Build



```
1 FROM ubuntu
2
3 RUN apt-get -y update
4 RUN apt-get -y install python3
5 RUN touch arquivo-de-boas-vindas.txt
```

Excluir uma única linha no Editor Nano
Ctrl+K

docker build -t ubuntu_python .

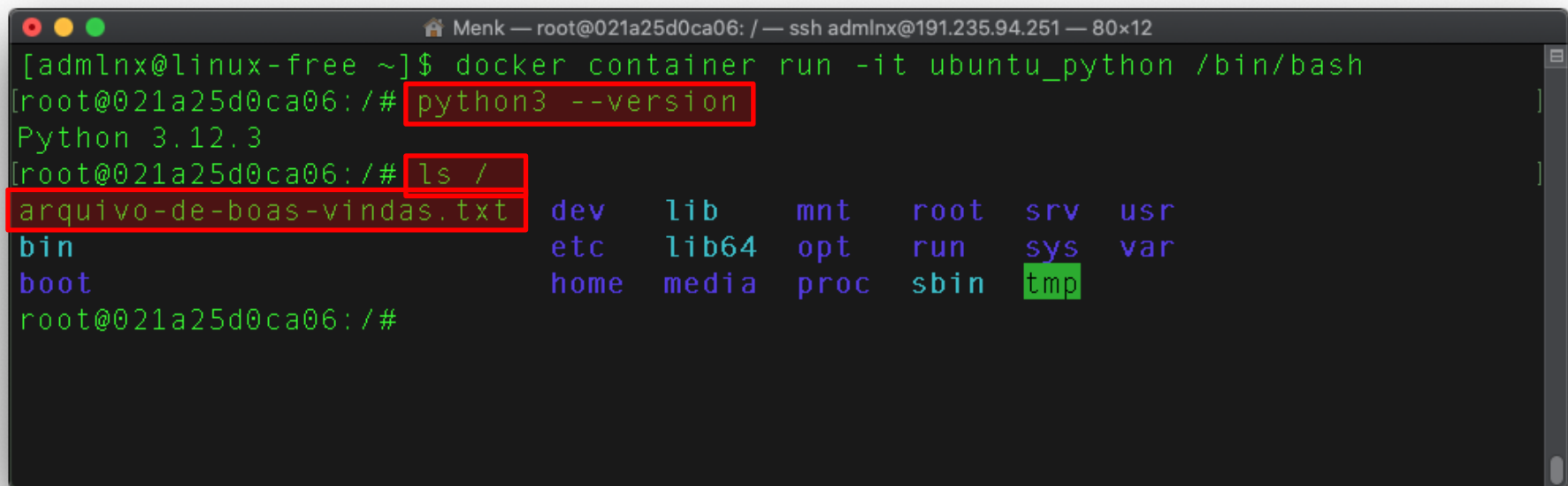
O comando BUILD consegue reutilizar diversas camadas e isso torna o processo muito mais rápido

```
Menk — root@4678c2fee316: / — -bash — 82x19
[iMac:~ Menk$ docker build -t ubuntu_python .
[+] Building 2.8s (8/8) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 145B 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 2.1s
=> [1/4] FROM docker.io/library/ubuntu@sha256:4b1d0c4a2d2aaf63b37111f34eb9 0.0s
=> CACHED [2/4] RUN apt-get -y update 0.0s
=> CACHED [3/4] RUN apt-get -y install python3 0.0s
=> [4/4] RUN touch arquivo-de-boas-vindas.txt 0.2s
=> exporting to image 0.4s
=> => exporting layers 0.4s
=> => writing image sha256:753d7b3e2600adb42da2257b20e548d19e798894d606ed9 0.0s
=> => naming to docker.io/library/ubuntu_python 0.0s

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
iMac:~ Menk$
```

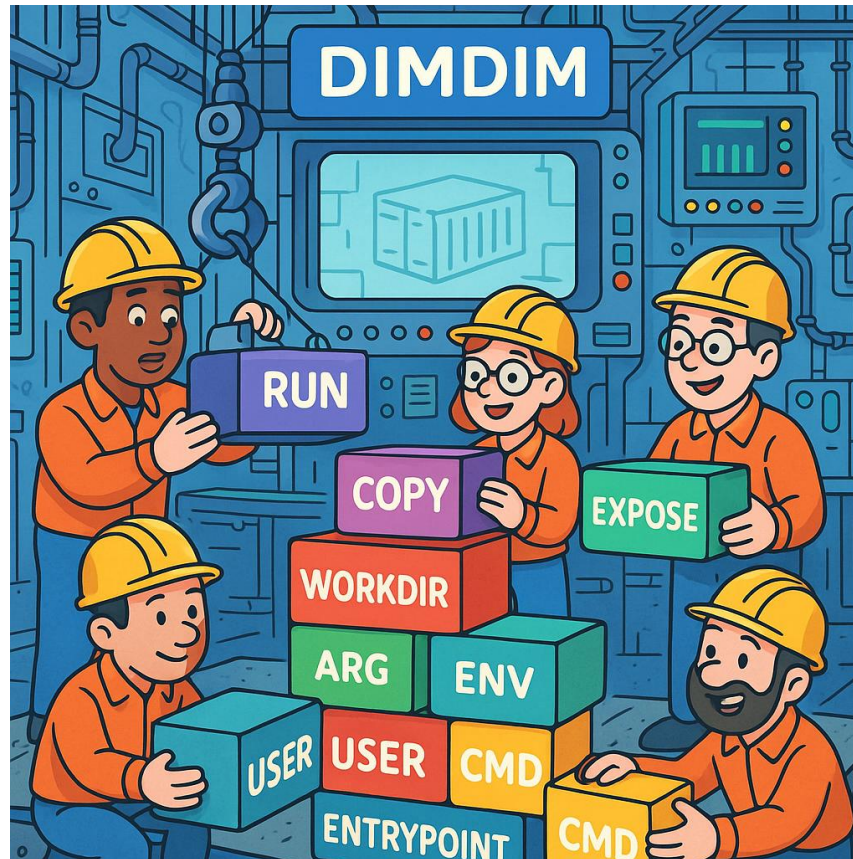
Executando um novo Container

```
docker container run --rm -it ubuntu_python /bin/bash
```

A terminal window titled 'Menk — root@021a25d0ca06: / — ssh adminx@191.235.94.251 — 80x12'. The terminal shows the execution of a Docker container. The first command is '[admlnx@linux-free ~]\$ docker container run -it ubuntu_python /bin/bash'. The prompt changes to '[root@021a25d0ca06:/#]'. The second command is 'python3 --version', which outputs 'Python 3.12.3'. The third command is 'ls /', which outputs a directory listing. The listing shows 'bin', 'boot', and 'root@021a25d0ca06:/#' on the first column. The second column contains 'dev', 'etc', and 'home'. The third column contains 'lib', 'lib64', and 'media'. The fourth column contains 'mnt', 'opt', and 'proc'. The fifth column contains 'root', 'run', and 'sbin'. The sixth column contains 'srv', 'sys', and 'tmp'. The seventh column contains 'usr' and 'var'. The 'tmp' directory is highlighted in green. The 'arquivo-de-boas-vindas.txt' file is highlighted in red in the first column.

```
[admlnx@linux-free ~]$ docker container run -it ubuntu_python /bin/bash
[root@021a25d0ca06:/#] python3 --version
Python 3.12.3
[root@021a25d0ca06:/#] ls /
arquivo-de-boas-vindas.txt  dev    lib    mnt    root   srv    usr
bin                          etc    lib64  opt    run    sys    var
boot                         home   media  proc   sbin   tmp
root@021a25d0ca06:/#
```

Os próximos comandos do Dockerfile estão na apostila **somente para referência e exercícios extras individuais**. Iremos realizar Hands-on com todos esses comandos trabalhando em conjunto para uma solução



- Exemplos de outros comandos no Dockerfile

ADD / COPY

Faz a cópia de um arquivo, diretório ou até mesmo fazer o download de uma URL de nossa máquina host e inserir dentro da imagem (ADD)

```
FROM ubuntu:18.04
```

```
RUN apt-get update -y
```

```
RUN apt-get install npm -y
```

```
ADD Dockerfile /root/arquivo-host-transferido.txt
```

➡ Direção: Host -> Container

docker build -t teste .

docker container run --name testeadd -it teste /bin/bash

Execute o comando `npm -v` para verificar a instalação, e após, `exit`, para sair do Container

- Exemplos de outros comandos no Dockerfile

EXPOSE

Informa quais portas a aplicação espera receber conexões

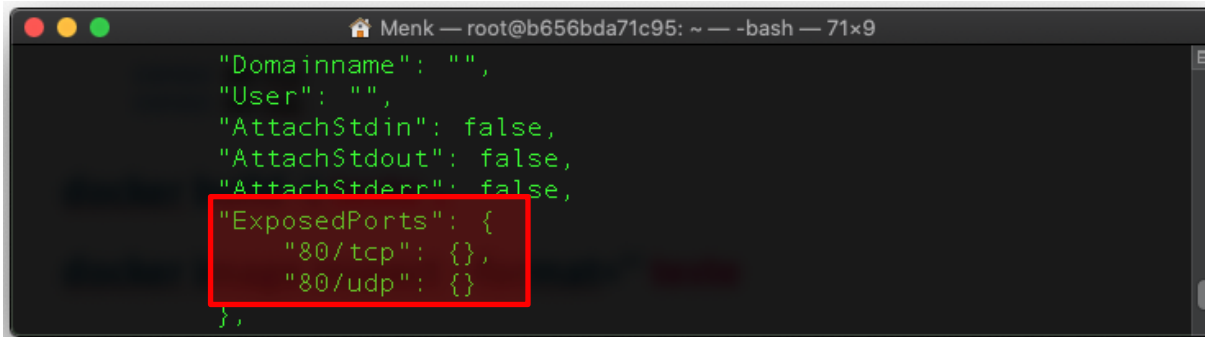
```
FROM ubuntu
```

```
EXPOSE 80/tcp
```

```
EXPOSE 80/udp
```

docker build -t teste .

docker image inspect teste



```
Menk — root@b656bda71c95: ~ — -bash — 71x9
"Domainname": "",
"User": "",
"AttachStdin": false,
"AttachStdout": false,
"AttachStderr": false,
"ExposedPorts": {
  "80/tcp": {},
  "80/udp": {}
},
```


- Exemplos de outros comandos no Dockerfile

EXPOSE

É importante entender que a instrução EXPOSE atua apenas como uma plataforma de informações (como Documentação) entre o criador da imagem Docker e o indivíduo que executa o Container

Esse comando não faz a publicação da porta

- ✓ Podemos usar o protocolo TCP ou UDP para expor a porta (Padrão TCP)
- ✓ Não mapeia portas na máquina Host
- ✓ Deve ser utilizado com o sinalizador de publicação (**-p**) ao criar o Container (**docker run**)

- Exemplos de outros comandos, **via DOCKER RUN**

EXPOSE

Exemplo com sinalizador de publicação

docker run -d --name nginx-server -p 80:80 nginx

↪ Direção: Host -> Container

```
Menk — root@52aca4c95fd0: / — -bash — 80x14
iMac:~ Menk$ docker run -d --name nginx-server -p 80:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
a603fa5e3b41: Pull complete
c39e1cda007e: Pull complete
90cfefba34d7: Pull complete
a38226fb7aba: Pull complete
62583498bae6: Pull complete
9802a2cfdb8d: Pull complete
Digest: sha256:e209ac2f37c70c1e0e9873a5f7231e91dcd83fdf1178d8ed36c2ec09974210ba
Status: Downloaded newer image for nginx:latest
23021825f5d60a42b1285d04e3d2500c3e3f101b129e8f950b5d29f047d1d143
iMac:~ Menk$
```

Acesse seu Web Browser em -> **localhost:80** ou **ipVM:80**

docker container rm -f nginx-server

- Exemplos de outros comandos no Dockerfile

WORKDIR

Define o ambiente de trabalho no Container, onde as instruções CMD, RUN, ENTRYPOINT, ADD etc executarão suas tarefas, além de definir o diretório padrão que será aberto ao executarmos o Container no modo Interativo (-it)

Crie o arquivo **no seu Host**, no Terminal execute:

```
echo { "nome": "Robert Plant", "banda": "Led Zeppelin" } > arquivo-host.json
```

```
FROM ubuntu:18.04
```

```
WORKDIR /app-java
```

```
COPY arquivo-host.json arquivo-host-transferido.json
```

```
docker build -t teste .
```

```
docker container run --name testeworkdir -it teste bash
```

- Exemplos de outros comandos no Dockerfile

CMD

- ✓ Usado para definir um comando padrão que é executado assim que você roda o Container, **não no Build da Imagem**
- ✓ No caso de vários comandos CMD, apenas o último é executado

```
FROM alpine
```

```
CMD ["echo", "Rodei na execução"]
```

```
CMD = bash -c
```

```
docker build -t testecmd .
```

```
docker container run --rm testecmd
```

- Exemplos de outros comandos no Dockerfile

ENTRYPOINT

É um ponto de entrada para seu Container, o que ele irá fazer ao iniciar. Permite que você configure **um Container que será executado como um executável**

```
FROM alpine
ENTRYPOINT ["top", "-b"]
```

docker build -t ptoentrada .

docker container run --rm ptoentrada

docker container run --rm ptoentrada -n 3

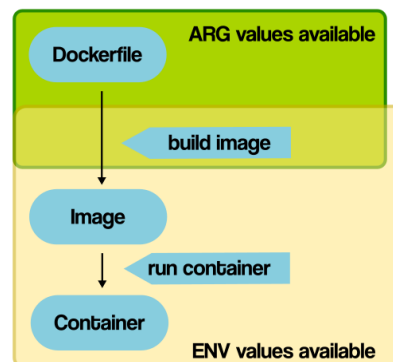
Envia parâmetros ao ENTRYPOINT. Executa:
-b -n 3

Documentação Linux: -n N -> Exit after N iterations

- Exemplos de outros comandos no Dockerfile

ARG e ENV

- ✓ A instrução **ARG** define uma variável que os usuários podem passar em tempo de compilação para o construtor da Imagem (comando **docker build**) usando o sinalizador: **--build-arg <varname>=<value>**
- ✓ **ARG** é a única instrução que pode preceder FROM no Dockerfile
- ✓ As variáveis de ambiente definidas usando a instrução **ENV** sempre substituem uma instrução **ARG** com o mesmo nome
- ✓ Ao contrário do **ARG**, as variáveis **ENV** também são acessíveis ao executar os Containers
- ✓ Os valores **ENV** também podem ser substituídos ao iniciar um Container (**-e** ou **--env-file**)



ARG nome # O ARG espera um valor
ARG nome=João # O ARG recebe um valor padrão
ENV estado=PB # O ENV recebe um valor padrão
ENV nome2=\$nome # O ENV recebe um valor padrão de um ARG
ENV nome2=\${nome} # O ENV recebe um valor padrão de um ARG

#fica
dica

NÃO PASSE VALORES DE SEGREDOS

Variáveis em tempo de compilação são visíveis através do comando **docker history**

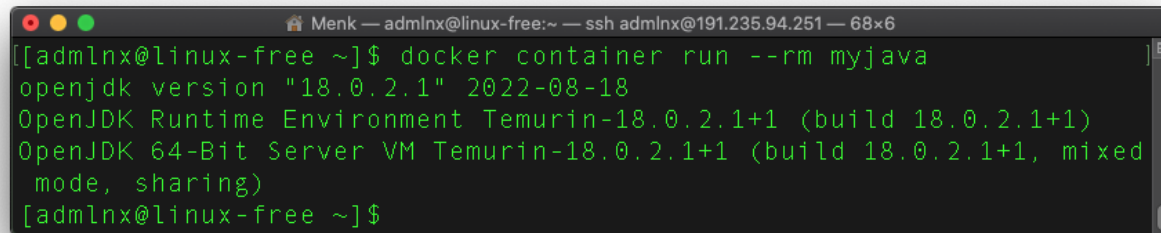
- Exemplos de outros comandos no Dockerfile

ARG

```
ARG JAVA_VERSION=17
FROM eclipse-temurin:${JAVA_VERSION}-jdk
CMD ["java", "--version"]
```

docker build -t myjava --build-arg JAVA_VERSION=18 .

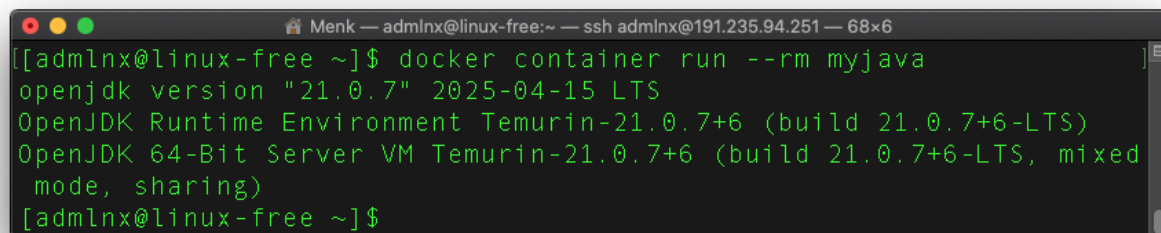
docker container run --rm myjava



```
Menk — admlnx@linux-free:~ — ssh admlnx@191.235.94.251 — 68x6
[admlnx@linux-free ~]$ docker container run --rm myjava
openjdk version "18.0.2.1" 2022-08-18
OpenJDK Runtime Environment Temurin-18.0.2.1+1 (build 18.0.2.1+1)
OpenJDK 64-Bit Server VM Temurin-18.0.2.1+1 (build 18.0.2.1+1, mixed
mode, sharing)
[admlnx@linux-free ~]$
```

docker build -t myjava --build-arg JAVA_VERSION=21 .

docker container run --rm myjava



```
Menk — admlnx@linux-free:~ — ssh admlnx@191.235.94.251 — 68x6
[admlnx@linux-free ~]$ docker container run --rm myjava
openjdk version "21.0.7" 2025-04-15 LTS
OpenJDK Runtime Environment Temurin-21.0.7+6 (build 21.0.7+6-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.7+6 (build 21.0.7+6-LTS, mixed
mode, sharing)
[admlnx@linux-free ~]$
```

- Exemplos de outros comandos no Dockerfile

ENV

Exemplo de utilização

```
FROM alpine
ENV hey="Olá"
ENV dir="/"
# ENV também pode ser utilizado na construção
ADD ./Dockerfile ${dir}
CMD echo $hey
```

docker build -t env-demo .

docker container run --rm env-demo

docker container run --rm -e hey="Salve" env-demo

docker container run --rm -it env-demo sh

Verifique o arquivo copiado

- Exemplos de outros comandos no Dockerfile

USER

Altera o usuário que irá executar os comandos

```
FROM node:alpine
```

```
RUN adduser -h /home/appuser -s /bin/sh -D appuser
```

→ Criação de um novo usuário

```
USER appuser
```

→ Utilização de um usuário já existente

```
RUN whoami
```

```
RUN touch /home/appuser/teste.txt
```

```
USER node
```

→ Utilização de um usuário já existente

```
RUN whoami
```

```
RUN touch /home/node/teste.txt
```

#Executar mais de um comando com a instrução CMD

```
CMD ls -l /home/appuser; ls -l /home/node
```

```
#CMD ["sh", "-c", "ls -l /home/appuser ; ls -l /home/node"]
```

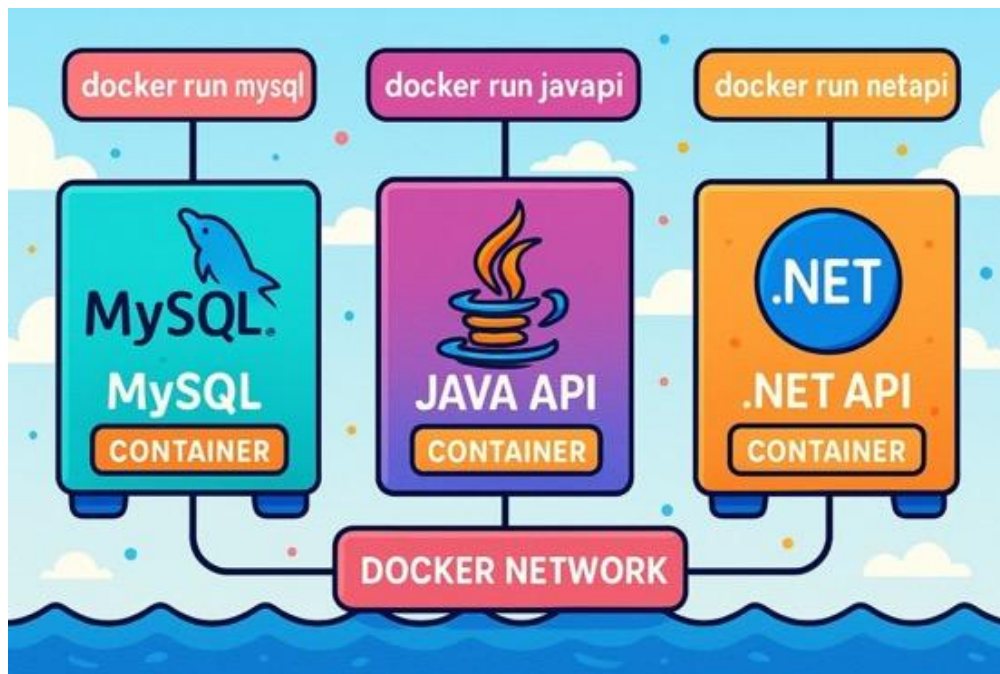
docker build -t user-demo .

docker container run --rm user-
demo

Cenário: Sistema de Transações Bancárias DimDim

O seu **objetivo** é **incluir o Sistema de Transações Bancárias DimDim**, composto por dois serviços: Banco de Dados MySQL e API Java Spring em **Dockefiles**, incluindo uma terceira API em .NET na solução completa

Isso irá permitir um gerenciamento simplificado, eliminar comandos manuais, diminuir o tamanho das imagens, 100% de reprodução etc.





01) Criar a estrutura do Projeto para o MySQL

Em seu diretório **HOME** realize o Clone do Repositório abaixo

git clone <https://github.com/profjoaomenk/mysql-dimdim.git>

02) Entre no diretório do Projeto MySQL

cd mysql-dimdim

03) Crie o arquivo do Dockerfile (Dockerfile.mysql):

-- Linux/Mac

touch Dockerfile.mysql

-- Windows (CMD)

copy nul Dockerfile.mysql

-- Windows (PowerShell)

> .\Dockerfile.mysql

04) Edite o arquivo criado Dockerfile.mysql

nano Dockerfile.mysql



05) Copie os comandos abaixo:

```
FROM mysql:8.0
```

```
ENV MYSQL_ROOT_PASSWORD=senha-dimdim
```

```
ENV MYSQL_DATABASE=db-dimdim
```

```
ENV MYSQL_USER=user-dimdim
```

```
ENV MYSQL_PASSWORD=senha-dimdim
```

```
EXPOSE 3306
```

```
COPY docker-entrypoint-initdb.d/init.sql /docker-entrypoint-initdb.d/init.sql
```

```
docker run --name mysql-dimdim -d \  
-e MYSQL_ROOT_PASSWORD=senha-dimdim \  
-e MYSQL_DATABASE=db-dimdim \  
-e MYSQL_USER=user-dimdim \  
-e MYSQL_PASSWORD=senha-dimdim \  
--network dimdim-network \  
-p 3306:3306 \  
-v mysql-dimdim-data:/var/lib/mysql \  
-v $(pwd)/docker-entrypoint-initdb.d/init.sql:/docker-entrypoint-initdb.d/init.sql mysql:8.0
```

06) Build da Imagem Personalizada

```
docker build -f Dockerfile.mysql -t mysql-dimdim .
```

O script init.sql agora está dentro da imagem, retirando a dependência do sistema operacional (compartilhar diretório)

07) Criação do Volume

```
docker volume create mysql-dimdim-data
```

08) Criação da Rede

```
docker network create dimdim-network
```



09) Execução do Container

```
docker run --name mysql-dimdim -d \
--network dimdim-network \
-p 3306:3306 \
-v mysql-dimdim-data:/var/lib/mysql \
mysql-dimdim
```

```
docker run --name mysql-dimdim -d \
-e MYSQL_ROOT_PASSWORD=senha-dimdim \
-e MYSQL_DATABASE=db-dimdim \
-e MYSQL_USER=user-dimdim \
-e MYSQL_PASSWORD=senha-dimdim \
--network dimdim-network \
-p 3306:3306 \
-v mysql-dimdim-data:/var/lib/mysql \
-v $(pwd)/docker-entrypoint-initdb.d/init.sql:/docker-entrypoint-initdb.d/init.sql mysql:8.0
```

10) Verificação do Banco MySQL

```
docker logs -f mysql-dimdim
```

```
docker exec -i mysql-dimdim mysql -uuser-dimdim -p'senha-dimdim' db-dimdim -e "SELECT * FROM transacoes;"
```

01) Realizar o Clone da API Java

Vá para seu diretório HOME e depois clone o projeto

cd

git clone <https://github.com/profjoaomenk/transacoes-api.git>

02) Entre no diretório do Projeto da API

cd transacoes-api

03) Dockerfile (Dockerfile.api):

-- Linux/Mac

touch Dockerfile.api

-- Windows (CMD)

copy nul Dockerfile.api

-- Windows (PowerShell)

> .\Dockerfile.api

04) Edite o arquivo criado Dockerfile.api

nano Dockerfile.api

05) Copiar o conteúdo abaixo:

Estágio 1: Build da aplicação com Maven e aproveitamento de cache

FROM maven:3.9-eclipse-temurin-17 AS builder

WORKDIR /app

Copie pom.xml e baixe dependências primeiro (cache eficiente)

COPY pom.xml .

RUN mvn dependency:go-offline

Agora copie o código (apenas se necessário para rebuild)

COPY src ./src

Compile o projeto

RUN mvn clean package -DskipTests

Estágio 2: Imagem enxuta para execução

FROM eclipse-temurin:17-jre

Criar usuário não-root antes de copiar arquivos

RUN addgroup --system appuser && adduser --system --ingroup appuser appuser

WORKDIR /app

Copie só o jar final

COPY --from=builder /app/target/*.jar app.jar

RUN chown appuser:appuser app.jar

USER appuser

Exponha porta da aplicação

EXPOSE 8080

Variáveis de ambiente (para sobrescrever facilmente)

ENV SPRING_DATASOURCE_URL=jdbc:mysql://mysql-dimdim:3306/db-dimdim \

SPRING_DATASOURCE_USERNAME=user-dimdim \

SPRING_DATASOURCE_PASSWORD=senha-dimdim

ENTRYPOINT ["java", "-jar", "app.jar"]

```
docker run --name api-dimdim -d \
-v $(pwd):/app \
-w /app \
-p 8080:8080 \
--network dimdim-network \
-e SPRING_DATASOURCE_URL=jdbc:mysql://mysql-dimdim:3306/db-dimdim \
-e SPRING_DATASOURCE_USERNAME=user-dimdim \
-e SPRING_DATASOURCE_PASSWORD=senha-dimdim \
maven:3.9-eclipse-temurin-17 \
bash -c "mvn clean package -DskipTests && java -jar target/*.jar"
```

Somente o último estágio (o último FROM) é usado para criar a imagem final que será produzida com o docker build, enquanto os estágios anteriores servem apenas como ambiente intermediário

06) Build da Imagem Personalizada

`docker build -f Dockerfile.api -t api-dimdim .`

07) Execução do Container

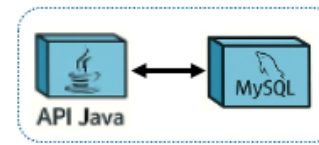
`docker run --name api-dimdim -d \`
`--network dimdim-network \`
`-p 8080:8080 \`
`api-dimdim`

```
docker run --name api-dimdim -d \
-v $(pwd):/app \
-w /app \
-p 8080:8080 \
--network dimdim-network \
-e SPRING_DATASOURCE_URL=jdbc:mysql://mysql-dimdim:3306/db-dimdim \
-e SPRING_DATASOURCE_USERNAME=user-dimdim \
-e SPRING_DATASOURCE_PASSWORD=senha-dimdim \
maven:3.9-eclipse-temurin-17 \
bash -c "mvn clean package -DskipTests && java -jar target/*.jar"
```

08) Verificação da API

`docker logs -f api-dimdim`

Aprendendo na Prática



FIAP

09) Testes da API com Persistência no Banco

```
curl -X GET http://localhost:8080/api/transacoes
```

```
curl -X POST http://localhost:8080/api/transacoes \  
-H "Content-Type: application/json" \  
-d '{  
  "descricao": "Compra no supermercado",  
  "valor": 150.75  
}'
```

```
curl -X PUT http://localhost:8080/api/transacoes/6 \  
-H "Content-Type: application/json" \  
-d '{  
  "descricao": "Compra no supermercado - ALTERADO",  
  "valor": 150.76,  
  "dataTransacao": "2024-06-18T00:00:00"  
}'
```

```
curl -X DELETE http://localhost:8080/api/transacoes/6
```

Vamos incluir uma API em .NET que irá utilizar um CRUD na mesma tabela do Banco MySQL

Objetivo: Criar container da API .NET com conectividade com o banco

- ✓ .NET
- ✓ Imagem pública para build e execução
- ✓ Operações CRUD no banco do exercício 1
- ✓ Testes de integração com CURL

Aprendendo na Prática



01) Realizar o Clone da API .NET

Vá para seu diretório HOME e depois clone o projeto

cd

git clone <https://github.com/profjoaomenk/ApiTransacoes.git>

02) Build Local – NÃO NECESSÁRIO EM NOSSO EXERCÍCIO

cd src/ApiTransacoes

dotnet build

```
Administrator: Command Prompt
C:\Users\labsfiap\ApiTransacoes>dotnet build
Welcome to .NET 8.0!
-----
SDK Version: 8.0.300

Telemetry
-----
The .NET tools collect usage data in order to help us improve your experience. It is collected by Microsoft and shared with the community. You can opt-out of telemetry by setting the DOTNET_CLI_TELEMETRY_OPTOUT environment variable to '1' or 'true' using your favorite shell.

Read more about .NET CLI Tools telemetry: https://aka.ms/dotnet-cli-telemetry

-----
Installed an ASP.NET Core HTTPS development certificate.
To trust the certificate, run 'dotnet dev-certs https --trust'
Learn about HTTPS: https://aka.ms/dotnet-https

-----
Write your first app: https://aka.ms/dotnet-hello-world
Find out what's new: https://aka.ms/dotnet-whats-new
Explore documentation: https://aka.ms/dotnet-docs
Report issues and find source on GitHub: https://github.com/dotnet/core
Use 'dotnet --help' to see available commands or visit: https://aka.ms/dotnet-cli

-----
Determining projects to restore...
  Restored C:\Users\labsfiap\ApiTransacoes\src\ApiTransacoes\ApiTransacoes.csproj (in 23.00 sec)
  ApiTransacoes -> C:\Users\labsfiap\ApiTransacoes\src\ApiTransacoes\bin\Debug\net8.0\ApiTransacoes.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:27.66
C:\Users\labsfiap\ApiTransacoes\src\ApiTransacoes>
```

03) Execução local – NÃO NECESSÁRIO EM NOSSO EXERCÍCIO

dotnet run

```
Administrator: Command Prompt - dotnet run
C:\Users\labsfiap\ApiTransacoes\src\ApiTransacoes>dotnet run
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\labsfiap\ApiTransacoes\src\ApiTransacoes
```

02) Automatizar com Dockerfile

Esse exercício já contempla o Dockerfile. Está no diretório **docker**, com o nome **Dockerfile**

Estando no diretório do Projeto (**api-transacoes**) crie a imagem personalizada com:

```
cd api-transacoes
```

```
docker build -f docker/Dockerfile -t api-transacoes .
```

03) Executar o Container com a API .NET

```
docker run --name api-transacoes -d \  
--network dimdim-network \  
-p 5000:8080 \    A API do Java Spring já escuta na porta 8080  
api-transacoes
```

04) Verificação da API

```
docker logs -f api-transacoes
```

05) Realizar os Testes de CRUD na API

Para recuperar linha(s) das tabela:

```
curl -X GET http://localhost:5000/api/transacoes
curl -X GET http://localhost:5000/api/transacoes/1
```

Para incluir uma linha na tabela:

```
curl -X POST http://localhost:5000/api/transacoes \
-H "Content-Type: application/json" \
-d '{
  "descricao": "Compra no supermercado(.NET)",
  "valor": 150.75
}'
```

```
docker exec -i mysql-dimdim mysql -uuser-dimdim -p'senha-dimdim' db-dimdim -e "SELECT * FROM transacoes;"
```

Para alterar a linha inserida na tabela:

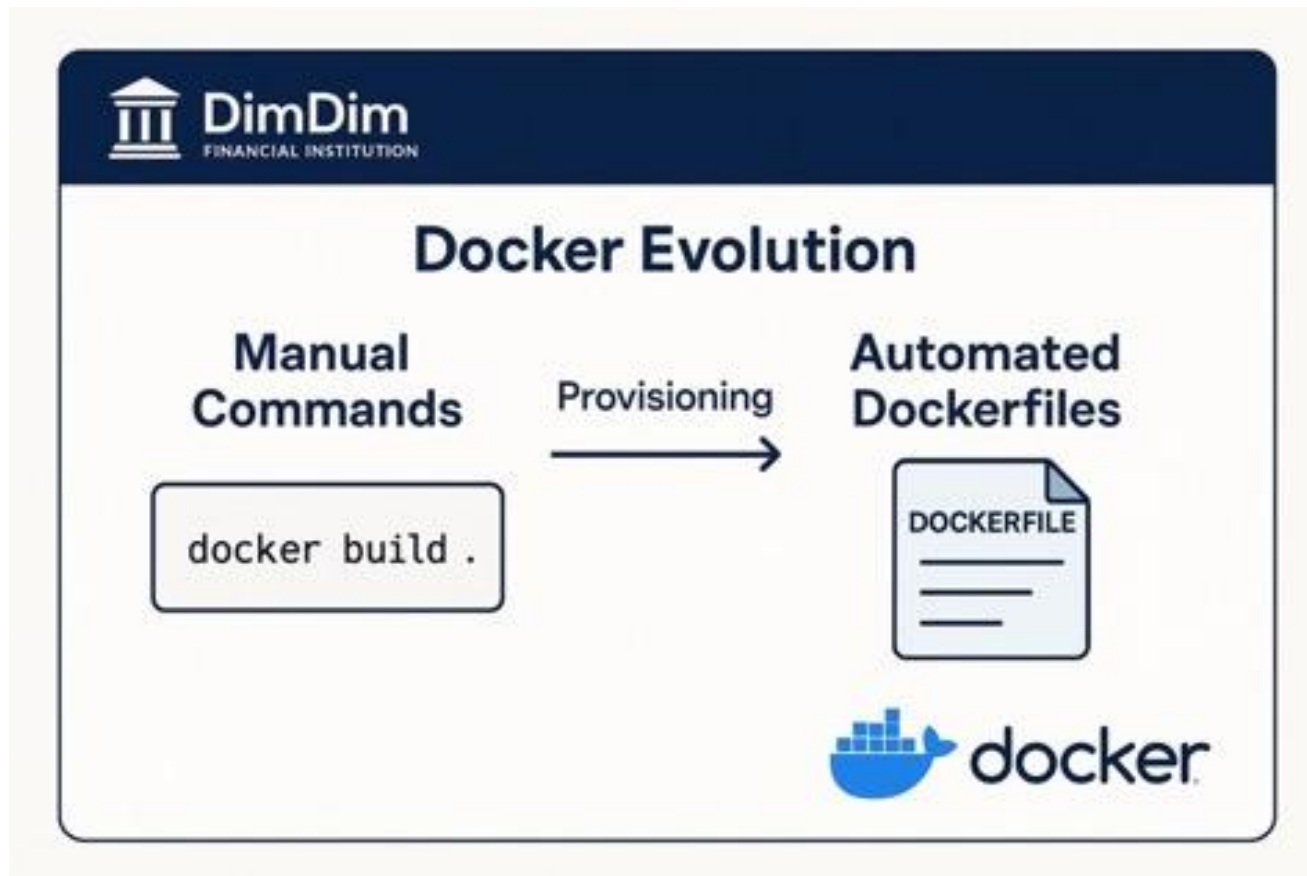
```
curl -X PUT http://localhost:5000/api/transacoes/7 \
-H "Content-Type: application/json" \
-d '{
  "id": 7,
  "descricao": "Compra no supermercado - ALTERADO",
  "valor": 150.76,
  "dataTransacao": "2024-06-18T00:00:00"
}'
```

Para deletar a linha inserida na tabela:

```
curl -X DELETE http://localhost:5000/api/transacoes/7
```

```
docker exec -i mysql-dimdim mysql -uuser-dimdim -p'senha-dimdim' db-dimdim -e "SELECT * FROM transacoes;"
```

A automação elimina a necessidade de comandos manuais repetitivos, introduzindo conceitos de Infrastructure as Code (IaC) e preparando o terreno para CI/CD e Pipelines



Evolução: Manual → Automatizado

Aspecto	Método Manual	Dockerfile Automatizado
Comandos	20+ comandos individuais	2 builds automatizados
Networking	Criação manual de rede	Configuração automática
Volumes	Criação e mount manual	Persistência automatizada
Variáveis	Passagem manual via -e	Definidas no Dockerfile
Build	Compilação externa	Multi-stage otimizado
Reprodutibilidade	Dependente de documentação	100% reproduzível
Tempo Deploy	15+ minutos manuais	3 minutos automatizados

Conclusão - Sistema DimDim Dockerizado

Benefícios Alcançados:

Automação Completa: Deploy em 3 minutos

Reprodutibilidade: Mesmo ambiente em qualquer máquina

Escalabilidade: Base para orquestração

Manutenibilidade: Código versionado e rastreável

Melhores Práticas Aprendidas:

Multi-stage builds para otimização

Redes Docker para microserviços

Volumes nomeados para persistência

Variáveis de ambiente para configuração

Melhores Práticas

- ✓ Use imagens oficiais e tags específicas
- ✓ Minimize camadas combinando comandos RUN
- ✓ Adote .dockerignore para reduzir contexto
- ✓ Crie usuário não-root e evite senhas no Dockerfile
- ✓ Utilize multi-stage





definitions

 **image** a static snapshot of container's configuration.

 **container** an application sandbox. each container is based on an image.

 **layer** image is composed of read-only file system layers. container creates single writable layer.

 **docker registry** remote server for storing Docker images

 **Dockerfile** a configuration file with build instructions for Docker images

 **docker engine** Docker platform installation running on a given host

 **docker client** client application that talks to local or remote Docker daemon

 **docker daemon** service process that listens to Docker client commands over local or remote network

 **docker host** server that runs Docker engine

 **volume** directory shared between host and container

docker run

docker run [OPTIONS] IMAGE[:TAG] [COMMAND]

Run a command in a new container.

metadata	--name=CONTNR_NAME Assign a name to the container.
	-l, --label NAME[=VALUE] Set metadata on the container.
process	-d, --detach Run in the background.
	-i, --interactive Keep STDIN open.
	-t, --tty Allocate a pseudo-TTY.
	--rm Automatically remove the container when process exits.
	-u USER Run as username or UID.
	--privileged Give extended privileges.
	-w DIR Set working directory.
	-e NAME=VALUE Set environment variable.
	--restart=POLICY Restart policy. no on-failure[:RETRIES] always unless-stopped
	-P, --publish-all Publish all exposed ports to random ports.
network	-p HOST_PORT:CONTNR_PORT Expose a port or a range of ports.
	--network=NETWORK_NAME Connect container to a network.
	--dns=DNS_SERVER1[,DNS_SERVER2] Set custom dns servers.
	--add-host=HOSTNAME:IP Add a line to /etc/hosts.
file system	--read-only Mount the container's root file system as read only.
	-v, --volume [HOST_SRC:]CONTNR_DEST Mount a volume between host and the container file system.
	--volumes-from=CONTNR_ID Mount all volumes from another container.

Dockerfile

```
FROM <image_id>
    base image to build this image from
RUN <command> shell form
RUN ["<executable>",
    "<param1>",
    ...,
    "<paramN>"]
    exec form
    executes command to modify container's file system state
MAINTAINER <name>
    provides information about image creator
LABEL <key>=<value>
    adds searchable metadata to image
ARG <name>[=<default value>]
    defines overridable build-time parameter:
    docker build --build-arg <name>=<value> .
ENV <key>=<value>
    defines environment variable that will be visible during
    image build-time and container run-time
ADD <src> <dest>
    copies files from <src> (file, directory or URL) and adds them
    to container file system under <dest> path
COPY <src> <dest> similar to ADD, does not support URLs
VOLUME <dest>
    defines mount point to be shared with host or other containers
EXPOSE <port>
    informs Docker engine that container listens to port at run-time
WORKDIR <dest>
    sets build-time and run-time working directory
USER <user>
    defines run-time user to start container process
STOPSIGNAL <signal>
    defines signal to use to notify container process to stop
ENTRYPOINT shell form or exec form
    defines run-time command prefix that will be added to all
    run commands executed by docker run
CMD shell form or exec form
    defines run-time command to be executed by default when
    docker run command is executed
```

LIMPAR O LABORATÓRIO DO DOCKERFILE

```
docker rm -f api-dimdim
```

```
docker rm -f api-transacoes
```

```
docker rm -f mysql-dimdim
```

```
docker system prune -a -f --volumes
```

```
cd
```

```
rm -rf mysql-dimdim
```

```
sudo rm -rf transacoes-api
```

```
sudo rm -rf ApiTransacoes
```





Copyright © 2026 Prof. João Menk

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).